Package Hallucinations and Skill-Based Verification

Vishesh
230129

Pranay Vishwakarma
230083

Abhishek Meena
230137

***Abstract*—This report studies package hallucinations, where a code-generating large language model (LLM) recommends or uses a non-existent package. Such errors create software supply-chain risk when an attacker registers a plausible missing name. M1 synthesizes the anchor study with five papers on code hallucination, agent Skills, harnesses, and tool-based correction. M2 supplies a feasible, API-based reproduction of two anchor conditions on a fixed 200-prompt Python subset. The long-term objective is an inference-time `SKILL.md` policy paired with a deterministic verification gate.**

***Index Terms*—package hallucination, large language models, code generation, software supply chain, agent skills, verification**

I. M1: Literature Survey

A. Research Area

The research area is **reliable and secure LLM-assisted software engineering**. The narrow focus is factual package/API claims in generated code: imports, installation commands, and direct package recommendations. Unlike a normal compiler error, a plausible missing dependency can become a package-confusion attack vector.

B. Selected Papers

Six works were selected; their key attributes are compared in Table I.

1) **Anchor:** Spracklen et al., "We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs" (USENIX Security 2025) [1].
2) Liu et al., "Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code" (HalluCode) [2].
3) Li et al., "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks" [3].
4) Han et al., "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?" [4].
5) Pan et al., "Natural-Language Agent Harnesses" [5].
6) Gou et al., "CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing" [6].

C. Initial Research Problem and Anchor Justification

Research question: Holding prompts, model, temperature, and repair budget fixed, does an executable `SKILL.md` plus deterministic package registry gate reduce invalid dependency references more than vanilla generation, static RAG, LLM self-detection, or prose-only instructions?

The anchor paper [1] is appropriate because it defines the exact security-relevant failure, includes code and prompt artifacts, tests model/temperature factors, and has RAG and self-detection baselines. HalluCode [2] shows functional correctness does not cover factual code errors. CRITIC [6] motivates external feedback. SkillsBench [3], SWE-Skills-Bench [4], and NLAH [5] motivate treating a harness as a measurable, ablatable component rather than unexamined controller logic.

II. M1: Reproducibility Checklist

The reproducibility status of the anchor study is summarized in Table II.

A. Data and Preprocessing

M2 uses 100 prompts from `Data/Python/SO_LY.json` and 100 from `Data/Python/LLM_LY.json`, sampled without replacement using seed `20261028`. Validity is assessed using the repository's `pypi_package_names.csv`, its false-positive exceptions, and `sys.stdlib_module_names`. Names are lowercased and normalized by replacing runs of `-`, `_`, and `.` with `-`.

B. Anchor Detection Channels

The anchor study uses three channels: (1) regular-expression extraction from explicit `pip install`/`npm install` commands; (2) an LLM query asking which dependencies are required by generated code; and (3) an LLM query asking which packages are useful for the original problem. The M2 code retains these ideas and adds AST import extraction with a regex fallback.

III. M2: Feasible Baseline Reproduction

A. Code and Experimental Setup

The submitted file `reproduce_m2_baseline.py` is a standalone, drop-in runner for a cloned anchor repository. It uses only the Python standard library plus an OpenAI-compatible endpoint; it can target OpenRouter, Groq, Gemini-compatible endpoints, or a locally hosted vLLM server. The setup is specified in Table III.

B. Metrics

For output i, let C_i denote deduplicated package claims and $H_i \subseteq C_i$ the claims missing from the PyPI snapshot, standard library set, and exception list. The hallucination rate is

$$\mathrm{HR} = 100\,\frac{\sum_i |H_i|}{\sum_i |C_i|}. \tag{1}$$

TABLE I
COMPARISON OF SELECTED WORKS

Work	Target / granularity	Mitigation	Key quantitative finding	Limitation / gap
Spracklen et al. (anchor) [1]	Non-existent PyPI/npm names; package, prompt, model, language, temperature.	Static RAG, LLM self-detection, fine-tuning.	576,000 samples, 16 models; 19.7% hallucinated recommendations overall; at least 5.2% commercial and 21.7% open-source.	No isolated deterministic gate; self-detection asks the generator to judge its own claim.
Liu et al. (HalluCode) [2]	Requirement, context, and knowledge conflict; fine-grained code snippets.	Prompt enhancement, self-refine, CoT, RAG.	1,134/3,120 samples (36.35%) contain hallucination; pass@1: 6.26 with vs. 54.73 without hallucination.	Broad taxonomy; no registry-based package verifier.
SkillsBench [3]	Procedural knowledge in agents; 86 tasks, 11 domains, 7,308 trajectories.	Curated vs. none vs. self-generated Skills.	Curated Skills: +16.2 percentage points (pp); self-generated: -1.3 pp.	Does not distinguish prose benefit from executable factual verification.
SWE-Skills-Bench [4]	Public Skills on repository tasks; paired requirement-level deltas.	Inject public Skills into agents.	3,920 paired evaluations; 92/98 Skills insignificant; pass ratio 76.6% to 76.9%.	Prose Skills can anchor agents to templates.
NLAH [5]	Harness logic; run-level policies and modules.	Natural-language harness with runtime contracts/gates.	Comparable outcomes to code/prompt harnesses, with inspectable shorter policy.	No package-validity task or comparison with registry evidence.
CRITIC [6]	Factual errors, flawed programs, toxicity; revision trajectory.	Tool-interactive verify–critique–revise.	TabMWP gains for LLaMA-2: +4.7% (7B) to +16.0% (70B).	Tool use is model-mediated; no package false-positive metric.

TABLE II
REPRODUCIBILITY CHECKLIST FOR THE ANCHOR STUDY

Checklist item	Status	Evidence / implication
Public dataset available?	Yes	`Data/Python/` and `Data/Javascript/` include `SO_AT`, `SO_LY`, `LLM_AT`, and `LLM_LY`.
Dataset size documented?	Yes	Full study: 576,000 generated samples over 16 models.
Public code available?	Yes	https://github.com/Spracks/PackageHallucination.
Public pretrained model available?	Partial	Repository expects checkpoints under `Models/`; not all evaluated checkpoints/serving setups are packaged.
Hardware requirement known?	Partial	Local code uses CUDA through Hugging Face; exact GPU/memory configuration is not fully specified.
Training time known?	Partial	Main study is inference, not training. README gives 24–72 hours for a full run.
Dependencies listed?	Yes	`environment.yml` and `requirements.txt` are supplied.
Evaluation metric clear?	Yes	Hallucination rate is invalid package references divided by total recommended references.
Feasible on Colab/Kaggle/free GPU?	Partial	Full replication is costly; a 200-prompt API or quantized small-model subset is feasible.
Reported results sufficiently reproducible?	Partial	Data/code are public, but detailed outputs, model/quantization versions, hardware, and registry timing constrain exact recreation.

The script also reports prompt-level hallucination incidence, false-positive refusal rate for C2 (valid claims that the LLM rejects), and mean regeneration count.

C. Original Result, Reproduced Result, and Gap

Table IV contrasts the original anchor result with the current state of the M2 reproduction.

D. Gap Analysis

If M2 differs from the anchor, the difference should be analyzed rather than hidden. Candidate sources include: random seed and subset, prompt source, API model/version drift, temperature/decoding, AST versus upstream extraction, registry age, import-name to distribution-name mismatch, regeneration implementation, and compute limits. The runner saves all configuration and prompt-level results required for this analysis.

E. Run Commands

```
export OPENAI_API_KEY='YOUR_KEY'
python reproduce_m2_baseline.py \
  --repo-root /abs/path/PackageHallucination \
  --base-url https://openrouter.ai/api/v1 \
  --models small=provider/model-8b \
           large=provider/model-70b
```

The data path can be validated without API usage:

```
python reproduce_m2_baseline.py \
  --repo-root /abs/path/PackageHallucination \
  --dry-run
```

After execution, the automatically generated result table can be included with:

```
\input{Tests/M2_API_Reproduction/summary.tex}
```

TABLE III
M2 REPRODUCTION SETUP. EXACT PROVIDER/MODEL IDENTIFIERS ARE SAVED BY THE RUNNER.

Item	Specification
Conditions	Two model sizes (small: 7B/8B; large: 70B+), $T \in \{0.0, 0.7\}$, and C0/C2.
C0	Vanilla code generation followed by extraction and deterministic registry scoring.
C2	Anchor self-detection: ask `Is [package] a valid Python package?`; reject flagged names and regenerate at most two times.
Model decoding	2,048 maximum generated tokens; package elicitation and binary self-detection run at temperature 0.
Preprocessing	Code-fence removal; AST imports; pip-command regex; recommendation parsing; de-duplication and normalization.
Training setup	None. This is an inference-time baseline.
Saved artifacts	Prompt-level CSV, summary CSV, Markdown table, LaTeX table, and `run_config.json`.

TABLE IV
ORIGINAL RESULT, REPRODUCED RESULT, AND GAP

Item	Result	Interpretation
Original result	19.7% hallucinated recommendations overall; at least 5.2% commercial and 21.7% open-source.	Full anchor-study result; not an expected exact value for M2.
Reproduced data/ preprocessing	Completed: fixed 200-prompt sample validated; 500,503 PyPI names loaded from the repository snapshot.	Confirms sample construction, registry loading, and repository integration.
Reproduced model result	Pending live endpoint execution.	No API key/model endpoint was supplied. A result must be generated with the provided code, not invented.
Reproduction gap	To be measured after run.	Compare only descriptively because M2 changes prompt size, model/provider, decoding implementation, and compute budget.

Generate → Extract imports/install/ recommendations → Normalize → Verify versioned registry → Pass or repair → Abstain after budget

Fig. 1. Verify-Skill harness pipeline for C4.

IV. M3/M4 PREVIEW: VERIFY-SKILL HARNESS

Hypothesis: a focused `SKILL.md` plus an executable AST/registry gate (C4) will reduce package hallucination rate relative to C0 (vanilla), C1 (static RAG), C2 (self-detection), and C3 (prose-only `SKILL.md`) at equal conditions. It should also reduce C2's false-positive refusals because validity is determined by external evidence rather than model self-belief.

The proposed pipeline is shown in Fig. 1.

The ablation plan removes prose while retaining the gate, removes the gate while retaining prose, varies repair budget, varies registry freshness, compares AST/regex/union extraction, and removes import aliases. The primary outcome is reference-level hallucination rate. Secondary outcomes are prompt incidence, valid-package recall, false-positive refusal rate, task utility, latency, and cost.

V. CONCLUSION

M1 identifies a defensible research gap: package self-detection is not deterministic external verification. The anchor is sufficiently public for a constrained but transparent reproduction. M2 supplies runnable code and clearly separates completed preprocessing replication from pending live-model results. M3/M4 will determine whether the executable verification gate, rather than prose alone, causes any reduction in hallucinations.

REFERENCES

[1] J. Spracklen *et al.*, "We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs," in *Proc. USENIX Security Symp.*, 2025. [Online]. Available: https://www.usenix.org/system/files/usenixsecurity25-spracklen.pdf

[2] F. Liu *et al.*, "Beyond functional correctness: Exploring hallucinations in LLM-generated code," arXiv:2404.00971. [Online]. Available: https://arxiv.org/abs/2404.00971

[3] X. Li *et al.*, "SkillsBench: Benchmarking how well agent skills work across diverse tasks," arXiv:2602.12670, 2026. [Online]. Available: https://arxiv.org/abs/2602.12670

[4] T. Han *et al.*, "SWE-Skills-Bench: Do agent skills actually help in real-world software engineering?" arXiv:2603.15401, 2026. [Online]. Available: https://arxiv.org/abs/2603.15401

[5] L. Pan *et al.*, "Natural-language agent harnesses," arXiv:2603.25723, 2026. [Online]. Available: https://arxiv.org/abs/2603.25723

[6] Z. Gou *et al.*, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2305.11738